

Comparaison expérimentale des algorithmes de chiffrement symétrique



Performance et robustesse : architecture x86 (Windows) vs ARM (Raspberry Pi)

Agenda



01

Nature du système & architecture



02

Protocole expérimental



03

Validation fonctionnelle — KAT



04

Résultats de performance



05

Robustesse cryptographique



06

Synthèse & recommandations



Nature du système

● Moteur expérimental

Aucune interface utilisateur — mesures uniquement

● Aucune interface graphique

Paramètres en entrée, fichiers CSV en sortie

● Conçu pour isoler la variable matérielle

Code identique sur les deux plateformes pour garantir la comparabilité

Répétabilité garantie — PyCryptodome s'appuie sur les mêmes routines C compilées sur les deux OS. Le comportement binaire est identique quelle que soit la plateforme.

experiment.py

ExperimentController



EncryptionEngine

Orchestration et délégation



OperationMode

ECB · CBC · CTR · GCM · StreamMode



CipherPrimitive

AES · DES · 3DES · Twofish · ChaCha20



PyCryptodome

Routines C compilées natives



Protocole expérimental

	Ordinateur Portable x86	Raspberry Pi
CPU	Intel Core i5-10300H	ARM Cortex-A72
OS	Windows 11	Raspberry Pi OS
AES-NI	✓ Oui	✗ Non
RAM	~16 Go	4 Go
Contexte	Multitâche	Minimaliste

5

algorithmes

AES · DES · 3DES · Twofish · ChaCha20

5

paliers

tailles : 64 octets → 16 384 octets

100

répétitions

 $IC_{95} = 1,96 \times \sigma / \sqrt{n}$

n = 100 — seuil choisi où l'IC à 95 % se stabilise ; tout accroissement supplémentaire apporte un gain de précision marginal au regard du coût computationnel.

Pourquoi Python ?

Critère	Python	C++	Java
Performance brute	⚠ Moyenne	✓ Élevée	⚠ Moyenne
Biais d'optimisation	✓ Faible	✗ Compilateur	✗ JIT / JVM
Reproductibilité inter-plateformes	✓ Excellente	⚠ Variable	⚠ Variable
Accès primitives bas niveau	✓ Via PyCryptodome	✓ Direct	⚠ Limité
Encodage des données	✓ octets bruts	✓ Contrôlable	✗ JVM dépendant

C++ aurait introduit un biais de compilation : les performances mesurées refléteraient les optimisations du compilateur autant que l'algorithme lui-même. Python, via PyCryptodome, délègue aux mêmes routines C sur les deux plateformes — le compilateur est hors équation.

Les messages sont générés en octets bruts via `os.urandom()` — aucune conversion str/unicode, aucun biais d'encodage inter-plateforme possible.



Known Answer Tests — KAT

Avant toute mesure de performance : prouver que les implémentations sont correctes.

Algorithme	Mode	Référence NIST/RFC	Statut
AES-128	ECB	NIST FIPS 197	✓ PASS
AES-256	GCM	NIST SP 800-38D	✓ PASS
DES	ECB	NIST SP 800-67	✓ PASS
3DES	ECB / CBC	NIST SP 800-67	✓ PASS
ChaCha20	Stream	RFC 8439	✓ PASS
AES	CBC / CTR	NIST SP 800-38A	✓ PASS

Toutes les implémentations produisent les sorties exactes attendues par les standards, donc les résultats de performance qui suivent sont fiables.



AES — Advanced Encryption Standard

- Chiffrement par blocs de **128 bits**
- Clés : **128 / 192 / 256 bits** → 10, 12 ou 14 tours
- Chaque tour : **SubBytes → ShiftRows → MixColumns → AddRoundKey**
- Standard NIST depuis **2001** (Rijndael), universellement déployé

AES-NI

Instructions matérielles Intel/AMD depuis 2010

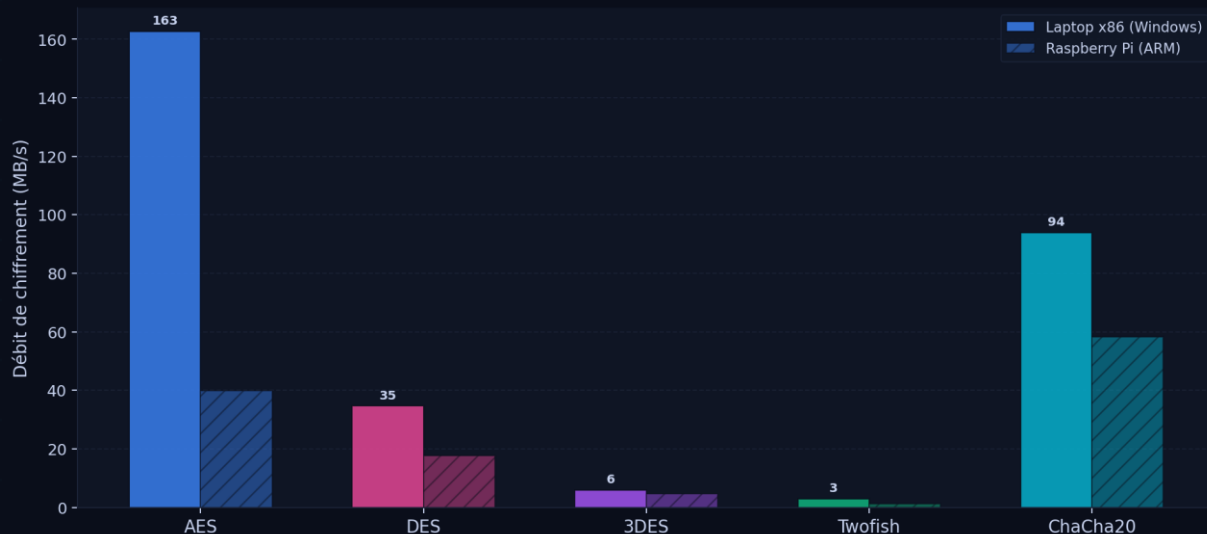
- 1 tour AES complet
= **1 cycle CPU**
- Impact mesuré :
x86 est 5,58× plus rapide que le Pi
- Sans AES-NI :
le chiffrement AES s'exécute entièrement en logiciel, instruction par instruction

→ L'écart x86 / Pi n'est pas attribuable à la vitesse du CPU, mais à la présence d'AES-NI.



Débit global : x86 vs Raspberry Pi

Comparaison 1 — Débit de chiffrement : Laptop x86 vs Raspberry Pi
(ECB sauf ChaCha20 · 4096 octets · meilleure clé par algorithme)



163 MB/s

AES sur x86

vs 46 MB/s sur Pi — écart dû à AES-NI exclusivement

3,54×

ratio AES x86 / Pi

le plus grand écart de tous les algorithmes testés

3 MB/s

Twofish sur x86

58× plus lent qu'AES — complexité du key schedule

AES sur x86 surpasse nettement les autres résultats, un écart attribuable non pas à la vitesse du CPU, mais à AES-NI. Sans cette instruction, le Pi effectuerait le même travail 3,54 fois plus lentement.

AES — L'écart x86 / Pi expliqué

Comparaison 2 — Rapport de performance x86 vs Raspberry Pi
(ECB sauf ChaCha20 · 4 096 octets · valeur > 1 = x86 plus rapide)



3,54x
ratio AES x86 / Pi

1,98x
ratio DES x86 / Pi

1,27x
ratio 3DES x86 / Pi

2,26x
ratio Twofish x86 / Pi

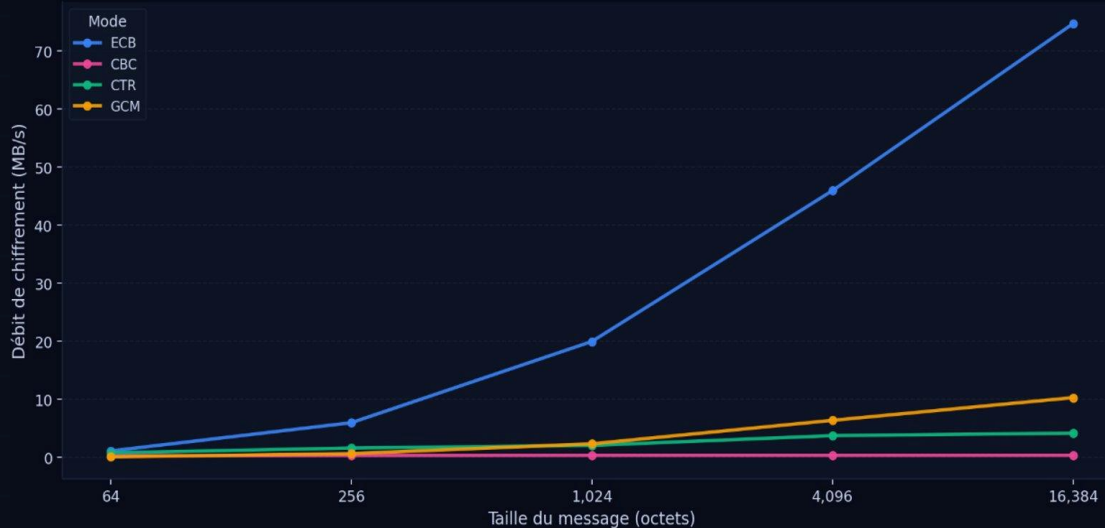
1,61x
ratio ChaCha20 x86 / Pi

Grâce à AES-NI sur x86, un tour AES complet s'exécute en un seul cycle CPU. Sans cette accélération matérielle, le Pi doit calculer entièrement en logiciel, ce qui produit un écart de 3,54x. Aucun autre algorithme testé ne bénéficie d'un tel levier matériel.



AES — Le piège des modes d'opération

Figure 3 — Comparaison des modes d'opération (AES-128)
(plateforme : laptop Windows x86)



ECB 163 MB/s

Rapide mais cryptographiquement cassé

GCM 25 MB/s

Recommandation 2026 — confidentialité + auth

CTR 6.6 MB/s

Parallélisable, bon compromis

CBC 0.73 MB/s

Chiffrement séquentiel — 223× plus lent qu'ECB

Avec 163 MB/s, ECB obtient le résultat le plus élevé du benchmark, et c'est pourtant le seul mode cryptographiquement cassé. Optimiser sans comprendre les implications en matière de sécurité constitue une faute professionnelle.



ChaCha20 — Conçu pour le logiciel pur

- **Chiffrement de flux**

pas de blocs — opère bit par bit sur le keystream

- **Clé unique : 256 bits**

pas de variantes 128/192 — simplicité par design

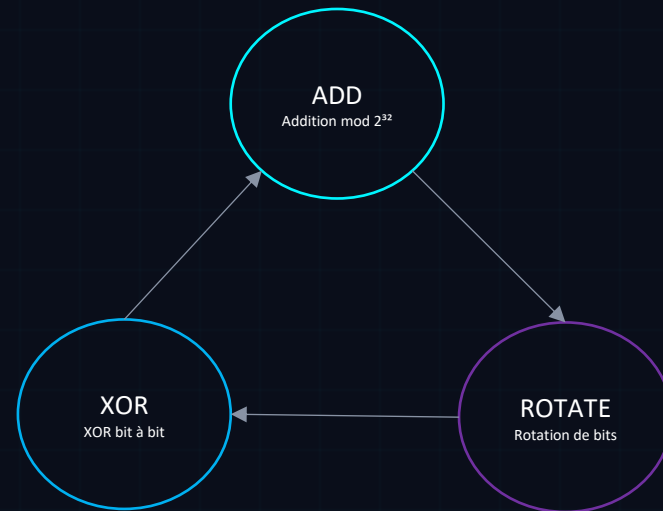
- **Opérations ARX**

Addition · Rotation · XOR — toutes natives au CPU

- **Daniel J. Bernstein, 2008**

optimisé pour ARM et x86 sans instructions dédiées

Cycle ARX — 20 tours



ChaCha20 est conçu pour s'exécuter en temps constant : les opérations ADD, ROTATE et XOR requièrent exactement un cycle CPU, indépendamment des données traitées. L'absence de table de substitution (S-Box) élimine tout risque d'attaque temporelle, ce qui explique l'adoption de ChaCha20 par Android et TLS 1.3 mobile.



ChaCha20 vs AES — Pourquoi ce choix existe

ChaCha20 a été conçu précisément pour corriger les faiblesses d'AES sur ARM.

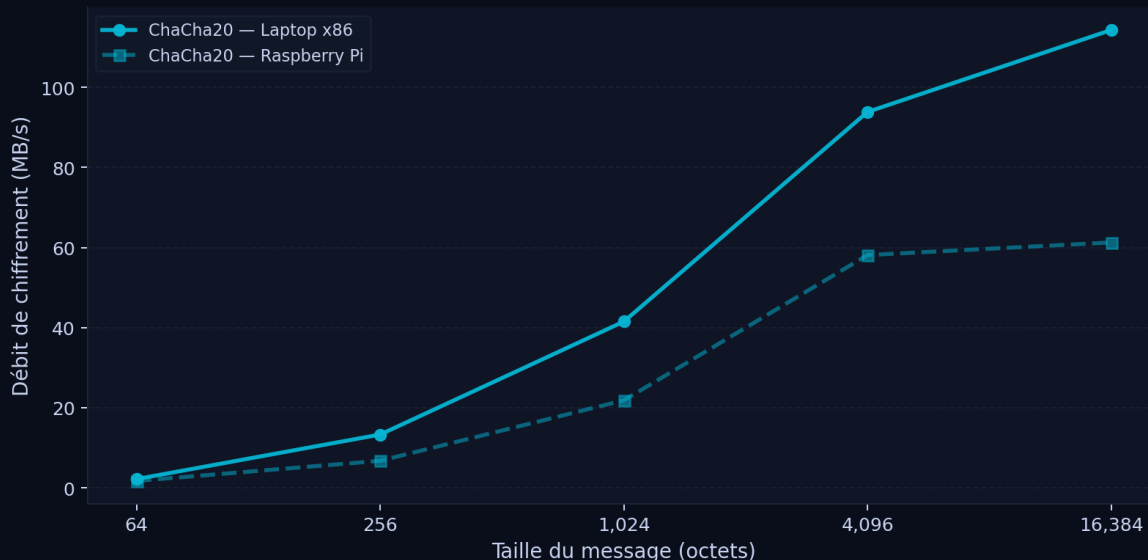
Critère	AES	ChaCha20
Opérations	S-Box (table lookup)	ARX — opérations natives
Accélération HW	AES-NI requis	Aucune nécessaire
Vulnérabilité timing	Possible sans AES-NI	Constant-time par design
Performance ARM	Pénalisé sans NI	Natif et efficace
Débit sur Pi (mesuré)	6.4 MB/s (GCM)	58 MB/s — 9.1× plus rapide
Usage standard	Serveur, cloud, x86	TLS 1.3, Android, IoT, ARM

Sur Raspberry Pi, ChaCha20 (58 MB/s) surpasse AES-GCM (6,4 MB/s) par un facteur de 9,1, ce qui en fait le choix évident pour tout déploiement IoT, mobile ou ARM. Il ne s'agit pas d'une simple alternative à AES, mais du standard sur architecture ARM.



ChaCha20 — L'algorithme qui refuse l'inégalité

Comparaison 5 — ChaCha20 : Laptop x86 vs Raspberry Pi
(aucune accélération matérielle sur les deux plateformes — ARX logiciel pur)



114 MB/s

ChaCha20 sur x86

vs 58 MB/s sur Pi — ratio 1.87× seulement

1.87×

ratio x86 / Pi

AES = 3.54× — ChaCha20 est 2× plus équitable

9.1×

avantage sur Pi

ChaCha20 (58 MB/s) vs AES-GCM Pi (6.4 MB/s)

Sur Raspberry Pi, ChaCha20 surpasse AES-GCM par un facteur de 9,1. Bernstein a conçu cet algorithme précisément pour ce scénario, soit un monde où tous les appareils ne disposent pas d'accélération cryptographique dédiée. ChaCha20 constitue d'ailleurs le standard sur Android et TLS 1.3 mobile.



DES, 3DES, Twofish — Pourquoi ils ont perdu

DES

MORT

40,7 MB/s

x86 · 18,3 MB/s Pi

Clé effective

56 bits

Cassé en

22h en 1998 (EFF)

Accélération HW

Aucune

Mort depuis 1998 — présent ici pour contexte historique.

3DES

DÉPRÉCIÉ

12,3 MB/s

x86 · 7,5 MB/s Pi

Sécurité effective

112 bits

Coût

3 passes séquentielles

NIST

Déprécié 2017, interdit 2023

Ni performant, ni suffisamment sécurisé

Twofish

OBSOLÈTE

2,82 MB/s

x86 · 1,29 MB/s Pi

Finaliste AES 1998

Perdu contre Rijndael

Problème

Key schedule trop complexe

vs AES

57× plus lent sur x86

Fascinant académiquement, mais jamais adopté industriellement.

Avec 163 MB/s contre 2,82 MB/s pour Twofish, soit un facteur de 57, ces trois algorithmes illustrent pourquoi AES s'est imposé comme standard. Leur intérêt ici est pédagogique, non opérationnel.

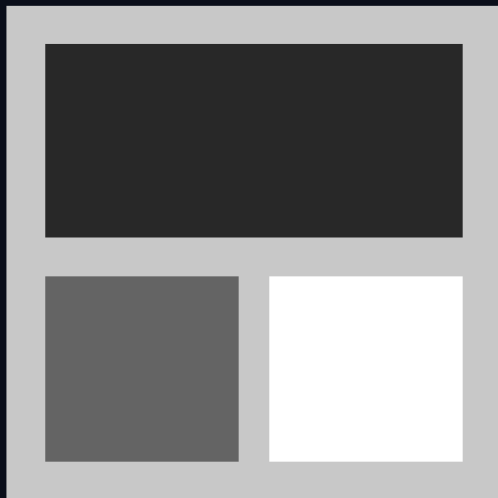


ECB — Le mode le plus rapide est le plus dangereux

Figure 8 — Vulnérabilité du mode ECB

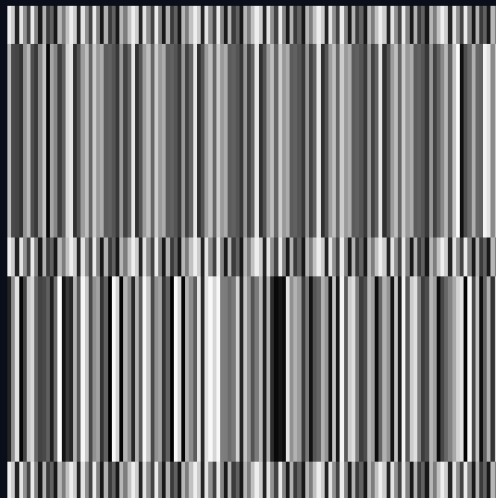
Les patterns de l'original sont préservés dans ECB — masqués dans CBC

(a) Image originale
(régions uniformes visibles)



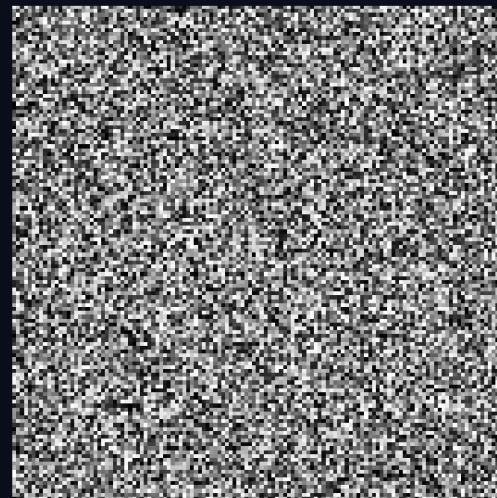
(b) Chiffré AES-ECB

⚠ Les patterns restent visibles !



(c) Chiffré AES-CBC

✓ Aucun motif structurel visible



ECB chiffre chaque bloc **indépendamment** avec la même clé

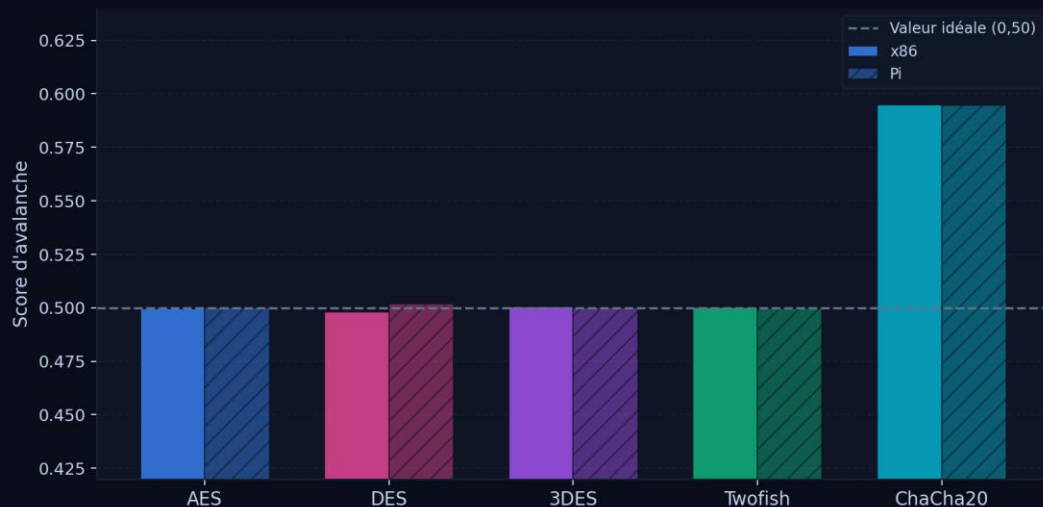
Deux blocs de plaintext identiques → **deux blocs de ciphertext identiques**

Les patterns structurels **survivent au chiffrement** — la clé n'a pas besoin d'être cassée

Avec 162 MB/s, ECB obtient le **résultat le plus élevé du benchmark**, et c'est pourtant le seul mode cryptographiquement cassé. Optimiser sans comprendre les implications en matière de sécurité constitue une faute professionnelle.

Effet d'avalanche — La santé cryptographique

Comparaison 4 — Effet d'avalanche : x86 vs Raspberry Pi
(propriété mathématique — les scores doivent être indépendants de la plateforme)



La valeur idéale est de 0,500 : modifier un seul bit en entrée doit changer exactement 50 % des bits en sortie. AES, DES, 3DES et Twofish atteignent tous cet idéal. Si ChaCha20 dépasse cette valeur, c'est en raison de son design, et non d'une faiblesse.

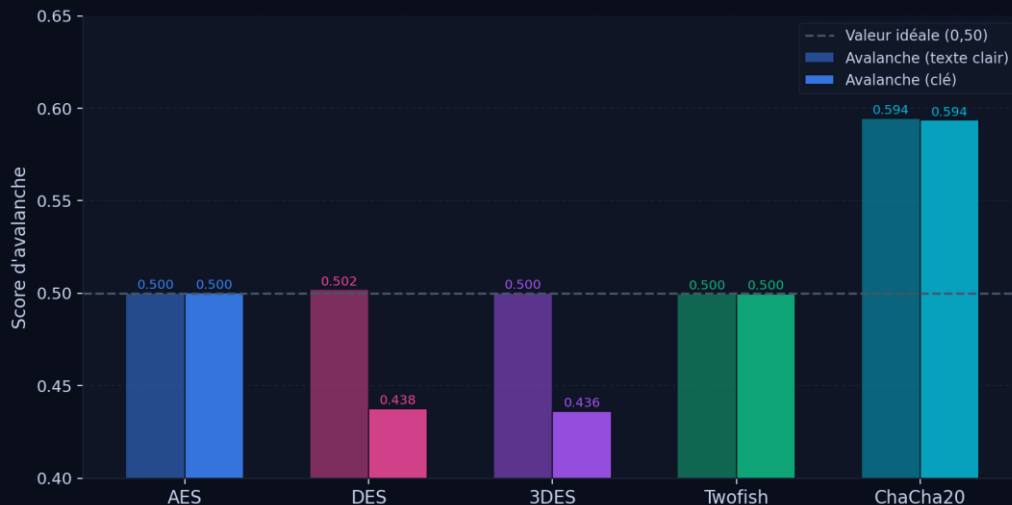
Algorithme	Score	Verdict
AES	0.4997	Parfait
3DES	0.5004	Excellent
Twofish	0.5004	Excellent
DES	0.4982	Acceptable
ChaCha20	0.5948	Anomalie

La valeur de 0,594 obtenue pour ChaCha20 ne constitue pas une faille, mais reflète une limite de la méthodologie employée. L'effet d'avalanche mesure la diffusion propre aux chiffrements par blocs, alors que ChaCha20 est un chiffrement de flux dont le keystream est calculé indépendamment du texte en clair.



Sensibilité aux clés — Quand le key schedule défaille

Figure 4b — Comparaison de l'effet d'avalanche : flip texte clair vs flip clé
(tous modes et tailles confondus)



Avec un score de 0,500 pour AES contre 0,438 pour DES, l'écart révèle que DES n'est pas seulement lent et cryptographiquement faible : son key schedule présente une déficience structurelle, et non un simple artefact de mesure..

Algorithme	Score clé	Verdict
AES	0.500	Parfait
Twofish	0.500	Excellent
ChaCha20	0.594	Voir sl.12
DES	0.438	Faible
3DES	0.436	Faible

Avec un score de 0,438, DES illustre un problème connu depuis les années 1990 : 64 clés dites « faibles » produisent des sorties structurellement moins aléatoires. 3DES hérite directement de cette faiblesse.



Stabilité — Le Raspberry Pi plus fiable que le laptop ?

Comparaison 6 — Stabilité des mesures : IC95 · Laptop x86 vs Raspberry Pi
(ECB · 4 096 octets · meilleure clé | valeur faible = mesures stables)



La vitesse brute ne garantit pas la fiabilité. Pour un protocole temps réel ou un module matériel de sécurité (HSM), la stabilité de la mesure importe autant que le débit. Le Raspberry Pi, bien que plus lent, se révèle plus prévisible — deux cas d'usage fondamentalement différents.

11,28 MB/s

IC95 AES sur x86

vs 2.06 MB/s sur Pi — le laptop est 5.5× plus erratique

≈ 0

IC95 Twofish

Stable sur les deux — trop lent pour que le bruit soit perceptible

6,38 MB/s

IC95 ChaCha20 x86

ARX sans matériel dédié = sensible au scheduler OS

Pourquoi cet écart ? Sur x86, Windows 11 introduit plusieurs sources de variabilité : Turbo Boost dynamique, processus en arrière-plan, et partage du cache L3. Le Raspberry Pi, sous Raspberry Pi OS, présente quant à lui un comportement déterministe par nature.



Synthèse — Comment choisir en 2026 ?



À éviter en toute circonstance : ECB, DES et 3DES. Quel que soit le contexte ou le niveau de performance observé, ces choix ne constituent pas des compromis acceptables en 2026.



Synthèse — Quel algorithme choisir en 2026 ?

Contexte	Algorithme	Mode	Raison
Serveur / cloud	AES-256	GCM	AEAD, NIST, TLS 1.3
IoT / ARM / Mobile	ChaCha20	Poly1305	ARX, constant-time
Données en masse	AES-256	CTR	Parallélisable
À éviter	DES, 3DES	—	Déprécié / cassé
INTERDIT	Tout algorithme	ECB	Patterns visibles

Conformes à NIST SP 800-175B et RFC 8446 (TLS 1.3)

AES-GCM sur x86, ChaCha20 sur ARM : voilà les deux recommandations à retenir. ECB, DES et 3DES sont à proscrire de tout nouveau développement.
L'ensemble de cette présentation constitue la preuve empirique de ces quelques principes.



Conclusion — Ce qui a été livré, ce qui vient ensuite

✓ TN3 — Livré

- 5 algorithmes implémentés et validés (KAT NIST)
- 4 modes d'opération (ECB, CBC, CTR, GCM) + StreamMode
- Expérimentations sur 2 plateformes : x86 Windows + Pi ARM
- ~600 mesures de débit collectées (n=100 par config.)
- Analyse : avalanche, sensibilité aux clés, stabilité CI95, ECB

→ TN4 — En préparation

- Analyse approfondie des hypothèses de recherche
- Comparaison avec la littérature (Stallings, NIST SP 800-175B)
- Rapport final — interprétation complète des résultats
- Conclusions sur les choix architecturaux
- Recommandations formelles

Le choix d'un algorithme de chiffrement ne relève pas uniquement de la sécurité, mais aussi d'une décision d'architecture, de plateforme et de contexte d'utilisation.